CHAPTER 38

Missing Indicator and Random Sample Imputation

Missingness Information, Distribution Preservation, Implementation

CampusX — 100 Days of Machine Learning
Feature Engineering Series, Videos 31–40

38.1 Missing Indicators

Sometimes the **fact that a value is missing** is itself predictive. For example, in a medical dataset, patients who did not report their income might systematically differ from those who did. A **missing indicator** is a binary feature (0 or 1) that records whether the original value was missing. This preserves the missingness information that would otherwise be destroyed by imputation.

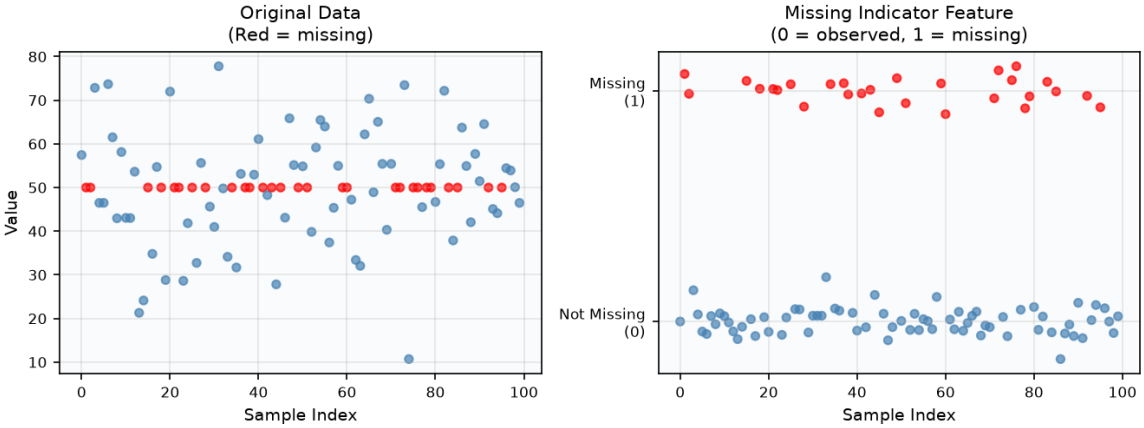


Figure 38.1: Left: original data with missing values (red dots). Right: the corresponding missing indicator feature — 1 where the original was missing, 0 where it was observed.

38.2 MissingIndicator in Scikit-learn

Parameter	Default	Description
features	"missing-only"	Which columns to generate indicators for: "missing-only" (only columns with NaN) or "all"
error_on_new	True	If True, raises error on unseen features at transform time

Table 38.1: MissingIndicator parameters.

```
import numpy as np
import pandas as pd
from sklearn.impute import MissingIndicator, SimpleImputer

df = pd.DataFrame({
'age': [25, 30, np.nan, 45, np.nan, 38, 52, np.nan],
'income': [50000, np.nan, 60000, np.nan, 75000, 80000, np.nan, 90000],
'score': [85, 92, 78, np.nan, 88, 95, 72, np.nan]
})

# Standalone MissingIndicator
indicator = MissingIndicator(features='missing-only')
indicator.fit(df)
missing_features = indicator.transform(df)

print("Missing indicator columns shape:", missing_features.shape)
print("Indicator for first 3 features:")
print(missing_features)

# ■■ add_indicator=True in SimpleImputer ■■
imputer = SimpleImputer(strategy='mean', add_indicator=True)
df_imputed = imputer.fit_transform(df)

print(f"\nOriginal shape: {df.shape}")
print(f"Imputed shape: {df_imputed.shape}")
print(f"(Added {df_imputed.shape[1] - df.shape[1]} indicator columns)")
print(f"\nImputed data:")
print(pd.DataFrame(df_imputed,
columns=list(df.columns) + ['age_indicator', 'income_indicator', 'score_indicator']))
```

Listing 38.1: MissingIndicator and add_indicator in SimpleImputer.

38.3 Random Sample Imputation

Random sample imputation fills each missing value by randomly selecting an observed value from the same feature. Unlike mean/median imputation, this preserves the original distribution of the feature because it draws from the actual observed values.

The algorithm is straightforward:

1. For each feature, collect all non-missing values.
2. For each missing value, randomly sample one value from the observed set (with replacement).
3. Replace the missing value with the sampled value.

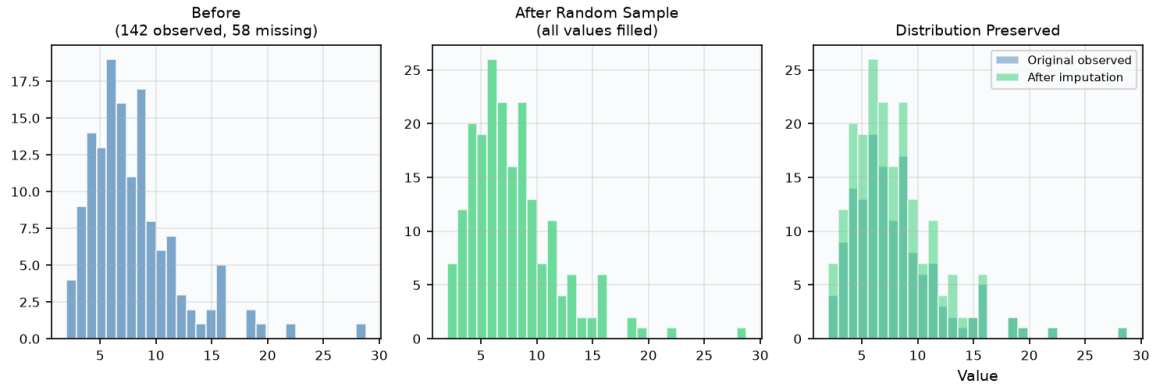


Figure 38.2: Random sample imputation preserves the distribution shape. Left: observed data before imputation. Centre: after random sample imputation. Right: the two distributions overlap closely.

38.3.1 Implementation

```
import numpy as np
import pandas as pd

def random_sample_imputation(df, random_state=42):
"""Fill missing values by sampling from observed values of the same column."""
np.random.seed(random_state)
df_imputed = df.copy()
for col in df.columns:
# Get observed (non-missing) values
observed = df[col].dropna().values
if len(observed) == 0:
continue # Skip all-missing columns
# Find missing positions
missing_mask = df[col].isna()
n_missing = missing_mask.sum()
# Sample with replacement from observed values
sampled = np.random.choice(observed, size=n_missing, replace=True)
# Fill
df_imputed.loc[missing_mask, col] = sampled
return df_imputed

# Example
np.random.seed(42)
df = pd.DataFrame({
'income': np.random.lognormal(10.5, 0.8, 200),
'age': np.random.normal(40, 12, 200),
'score': np.random.normal(70, 15, 200)
})
```

```
# Introduce missing values
for col in df.columns:
mask = np.random.random(200) < 0.2
df.loc[mask, col] = np.nan

print(f"Missing before: {df.isnull().sum().sum()}")

# Impute
df_filled = random_sample_imputation(df)
print(f"Missing after: {df_filled.isnull().sum().sum()}")

# Compare distributions
print("\n=== DISTRIBUTION COMPARISON ===")
print(f"{'Col':<10} {'Obs Mean':>10} {'Filled Mean':>12} {'Obs Std':>10} {'Filled Std':>11}")
for col in df.columns:
obs = df[col].dropna()
print(f"{col:<10} {obs.mean():>10.2f} {df_filled[col].mean():>12.2f} "
f"{obs.std():>10.2f} {df_filled[col].std():>11.2f}")
```

Listing 38.2: Random sample imputation implementation.

38.4 Advantages and Limitations

38.4.1 Missing Indicator

- **Advantage:** Preserves the information that a value was missing, which can be predictive in MAR/MNAR scenarios.
- **Advantage:** Easy to combine with any imputation method via add_indicator=True.
- **Limitation:** Adds extra features, increasing dimensionality. For datasets with many features and low missingness, this can add noise.
- **Limitation:** The indicator is only useful if the model can exploit it. Simple linear models may not benefit as much as tree-based models.

38.4.2 Random Sample Imputation

- **Advantage:** Preserves the original distribution (mean, variance, shape) better than mean/median imputation.
- **Advantage:** Simple to implement and understand.
- **Limitation:** Does not consider relationships between features. Each feature is imputed independently.
- **Limitation:** Introduces randomness — results vary across runs unless you fix the random seed.
- **Limitation:** Can create implausible combinations (e.g., imputing an income that does not match the education level of that observation).

Key Limitation of Random Sample Imputation
Because each feature is imputed independently, random sample imputation can create **internally inconsistent** records. An imputed age of 25 might be paired with an imputed retirement status of "retired" — a combination that would never occur in the real data. Multivariate methods like KNN and MICE (Chapters 39–40) address this by considering feature relationships.

38.5 Key Takeaways

> **★ Missing indicators preserve the information that a value was missing. Random sample imputation fills NaNs with randomly chosen observed values, preserving the marginal distribution. Both are useful additions to the imputation toolkit but have important limitations regarding feature relationships.**